

Project Report: Enhancing LLM Inference with GraphRAG

Daniel Granström
Aalto University
Espoo, Finland
daniel.granstrom@aalto.fi

Joonatan Korpela
Aalto University
Espoo, Finland
joonatan.korpela@aalto.fi

Mikael Siidorow
Aalto University
Espoo, Finland
mikael.siidorow@aalto.fi

Abstract

The course project is available at <https://github.com/dcgran/CS-E4780-project> [1].

- GraphRAG implementation on Nobel Laureates dataset
- Text2Cypher accuracy improvements + performance optimization
- Key metrics: accuracy, latency, cache hit rate, memory usage
- GitHub repository: <https://github.com/dcgran/CS-E4780-project> [1]

Keywords

Large Language Models, Retrieval-Augmented Generation

ACM Reference Format:

Daniel Granström, Joonatan Korpela, and Mikael Siidorow. 2025. Project Report: Enhancing LLM Inference with GraphRAG. In *Proceedings of CS-E4780 Course Project Evaluation Tables (CS-E4780)*. ACM, New York, NY, USA, 5 pages.

1 Introduction

Retrieval-Augmented Generation (RAG) systems enhance Large Language Model (LLM) inference by incorporating external knowledge sources to improve factual accuracy and reduce hallucinations. Traditional RAG approaches rely on semantic similarity search over unstructured text chunks, which works well for many tasks but struggles to capture the rich relational structure underlying complex domains such as scientific collaboration networks, enterprise knowledge bases, and multi-hop reasoning problems. GraphRAG addresses this limitation by integrating graph representations of knowledge into the retrieval process [2], enabling LLMs to reason over structured relationships and perform multi-hop queries that would be difficult to express through text-based retrieval alone.

This project implements a GraphRAG system using the Nobel Laureates dataset, which contains mentorship relationships spanning from the 16th century to 2021, enriched with biographical data from the official Nobel Prize API. The dataset is stored in Kuzu, a graph database that supports efficient pattern matching through the Cypher query language. The system pipeline consists of four main stages: schema pruning to identify relevant graph structure, Text2Cypher generation to translate natural language questions into Cypher queries, query execution against the graph database, and response generation by the LLM based on retrieved results.

We address two primary objectives in this work. First, we enhance the Text2Cypher component to improve query generation accuracy through three techniques: (1) dynamic few-shot exemplar selection based on semantic similarity to the input question, (2) a self-refinement loop that validates generated queries using

dry-run execution and repairs syntax errors iteratively, and (3) rule-based post-processing to enforce common patterns such as case-insensitive string matching. Second, we optimize system performance by implementing an LRU cache keyed on question and schema hashes, and conduct comprehensive benchmarking to characterize latency across all pipeline stages and quantify memory utilization.

Our evaluation includes an ablation study examining all meaningful combinations of Text2Cypher enhancements, quantifying the accuracy-latency trade-offs inherent in each approach. We measure both syntax correctness and semantic correctness against a diverse test set, and analyze cache performance under different workload patterns. The results provide practical insights into which optimizations yield the best balance between response quality and system efficiency.

The remainder of this report is organized as follows. Section 2 describes the system architecture and design decisions. Section 3 details the implementation of both tasks, including testbed setup and measurement methodology. Section 4 presents our experimental evaluation with comprehensive performance analysis. Section 5 discusses key findings and lessons learned, and Section 6 concludes.

2 System Architecture

2.1 GraphRAG Pipeline

Our GraphRAG system implements a five-stage pipeline that transforms natural language questions into structured graph queries and generates contextually grounded responses. Figure 1 illustrates the complete workflow.

Stage 1: Schema Retrieval. The system begins by extracting the complete graph schema from Kuzu, including all node types, relationship types, and their associated properties. This schema provides a comprehensive structural description of the knowledge graph that will be used in subsequent stages.

Stage 2: Schema Pruning. Given a user question, the system uses an LLM to identify which subset of the schema is relevant to answering that question. This pruning step reduces the context size for downstream stages, improving both accuracy and latency. The pruned schema is serialized and hashed to serve as part of the cache key, enabling reuse across questions that involve the same graph structure.

Stage 3: Text2Cypher Generation. The pruned schema and user question are passed to the Text2Cypher component, which generates a syntactically and semantically valid Cypher query. This stage incorporates three enhancement mechanisms (detailed in Section 3.2): similarity-based few-shot exemplar selection, self-refinement with validation feedback, and rule-based post-processing. The generated query is cached using a composite key of question hash and schema hash.

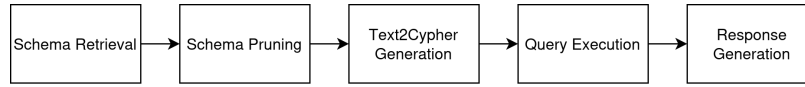


Figure 1: Flowchart depicting the complete workflow of the GraphRAG pipeline

Stage 4: Query Execution. The generated Cypher query is executed against the Kuzu database. Kuzu’s columnar storage and optimized query engine enable efficient pattern matching over the graph structure. The query results are returned as structured data (e.g., lists of nodes, relationships, or aggregated values).

Stage 5: Response Generation. Finally, an LLM synthesizes a natural language response based on the original question, the generated query, and the structured results from the database. This stage translates the retrieved facts into a coherent, human-readable answer that directly addresses the user’s information need.

2.2 Knowledge Graph Schema

The Nobel Laureates knowledge graph represents prize winners, their institutional affiliations, and geographic metadata. The graph is constructed from the official Nobel Prize API, which provides comprehensive biographical and institutional data for all laureates from 1901 to 2021 across four prize categories. Figure 2 illustrates the complete schema.

Node Types. The schema defines six entity types:

- **Scholar:** Represents Nobel Prize laureates. Properties include id, knownName, fullName, gender, birthDate, deathDate, and scholar_type (set to “laureate” for all nodes from the API data).
- **Prize:** Represents individual Nobel Prize awards in a given year and category. Properties include prize_id (composite key of year and category), awardYear, category (physics, chemistry, medicine, economics), dateAwarded, motivation, prizeAmount, and prizeAmountAdjusted.
- **Institution:** Academic and research institutions affiliated with scholars. Properties include name (standardized current name).
- **City:** Geographic locations representing cities associated with scholar birthplaces and institutional locations. Properties include name and state.
- **Country:** National-level geographic entities. Properties include name.
- **Continent:** Continental-level geographic entities for regional analysis. Properties include name.

Relationship Types. The schema defines seven relationship types connecting entities:

- **WON:** Connects a Scholar to a Prize they received. Includes a portion property indicating the share of the prize (e.g., “1/2” for shared prizes).
- **BORN_IN:** Connects a Scholar to their birth City.
- **DIED_IN:** Connects a Scholar to the City where they died (defined but sparsely populated in current data).
- **AFFILIATED_WITH:** Connects a Scholar to an Institution they were associated with at the time of receiving the prize.
- **IS_LOCATED_IN:** Connects an Institution to its host City.
- **IS_CITY_IN:** Connects a City to its Country.

- **IS_COUNTRY_IN:** Connects a Country to its Continent.

This schema enables multi-hop queries such as: “How many Physics laureates were born in cities located in European countries?” or “Which institutions affiliated with the most Chemistry prize winners are located in the United States?” The hierarchical geographic structure (City → Country → Continent) supports analysis at multiple spatial granularities, while the institutional affiliations enable studies of knowledge production across research organizations.

2.3 Component Interactions

The system is made up of three main components: the core Graph RAG pipeline that turns questions into answers, supporting modules for schema handling, few-shot retrieval, validation, caching, and post-processing, and separate benchmarking tools that measure accuracy, latency, and internal behavior.

The core Graph RAG pipeline handles the flow from a user question to a final answer. It first checks whether a question is relevant, prunes the graph schema to the parts that matter, generates a Cypher query, and optionally retries the query if it fails a validation check. Then, it runs the query in Kuzu and turns the results into a readable answer. Finally, the result post-processing fixes any formatting issues. The pipeline also caches schema pruning and query generation so repeated or similar questions run faster.

Supporting modules handle the details behind these steps. They load the database schema, manage connections, and define DSPy components for pruning, query generation, and answer creation. For few-shot prompting, the system uses a KNN retriever to pull in examples that guide the query generation. For the query validation, the system keeps retrying the query generation until the LLM produces a query that passes an EXPLAIN check. During the run, the system collects timing data, cache hits, and other metadata for later benchmarking.

This design is built around the concept of separation of concerns, which keeps each part of the system focused on a single job. The core GraphRAG module handles the overall question-to-answer flow, while the database manager takes care of connections and schema, and the DSPy components define the configurations for pruning, query generation, and answer creation. This separation keeps the core logic simple, makes the system easier to debug, and allows the evaluation scripts to run many configurations without touching the core pipeline. Strongly typed schema objects also help guide the LLM, which makes validation more predictable.

3 Implementation

3.1 Testbed Setup

Hardware Environment. Experiments were conducted on a machine with [TODO: CPU model], [TODO: RAM] GB of memory, running [TODO: OS]. However, hardware specifications have limited

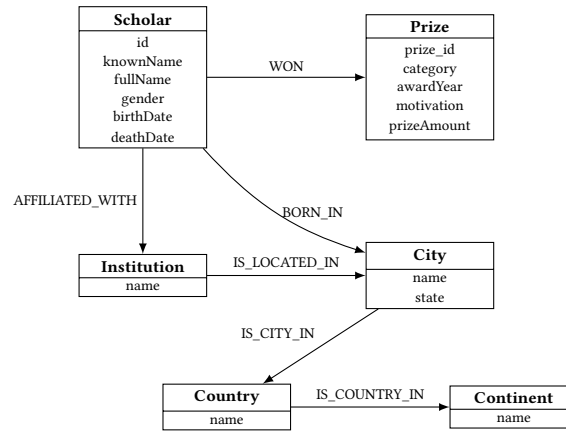


Figure 2: Knowledge graph schema showing node types and relationships in the Nobel Laureates dataset. The schema supports multi-hop queries across prize winners, their institutional affiliations, and hierarchical geographic locations.

impact on end-to-end latency in this system, as the LLM inference—which dominates execution time—is performed remotely via API calls rather than locally.

Software Environment. The system is implemented in Python 3.13 using the following key dependencies:

- **Kuzu** (v0.11.1): Embedded graph database supporting the Cypher query language, used for storing and querying the Nobel Laureates knowledge graph.
- **DSPy** (v3.0.1): Framework for building LLM-powered pipelines with structured prompts, used to orchestrate schema pruning, query generation, and answer synthesis.
- **Sentence Transformers** (v3.0.0): Provides the all-MiniLM-L6-v2 embedding model for computing semantic similarity in KNN-based few-shot exemplar selection.
- **cachetools** (v6.1.0): Implements the LRU cache for storing query generation results.

LLM Configuration. All LLM inference is performed via the OpenRouter API using the google/gemini-1.5-flash model. This remote inference setup means that observed latencies are influenced by network conditions and API response times, which introduces variability across runs. We mitigate this by averaging results over multiple executions.

Test Query Dataset. We evaluate the system using a benchmark suite of 10 natural language questions designed to cover diverse query patterns:

- **Entity lookup:** “Who won the Nobel Prize in Physics in 2020?”
- **Multi-hop with constraints:** “Which scholars won prizes in Physics and were affiliated with University of Cambridge?”
- **Aggregation:** “How many Nobel Prizes have been awarded in Chemistry?”
- **Geographic filtering:** “List all Nobel Prize winners from Sweden.”
- **String matching:** “Find all scholars whose name contains ‘Einstein’.”
- **Adversarial queries:** Questions designed to stress-test the system, such as “List Nobel laureates born in Sweden and located

in Paris” (requires correct join semantics) and “Which laureates winning literature prizes are affiliated with Stanford but not Cambridge?” (references non-existent prize category).

One query is duplicated to measure cache hit behavior. The adversarial queries help identify failure modes in query generation and schema understanding.

3.2 Task 1: Text2Cypher Enhancement

Dynamic Few-Shot Exemplar Selection. We implement dynamic exemplar selection using K-nearest neighbors (KNN) retrieval over a trainset of 15 curated question-query pairs covering diverse query patterns. At inference time, the user’s question is embedded using Sentence Transformers (all-MiniLM-L6-v2), and the $k = 3$ most similar examples are retrieved and included as few-shot context. This is implemented via DSPy’s KNNFewShot optimizer. We also support a static baseline that always selects the first k examples for comparison.

Self-Refinement Loop. We implemented a self-refinement loop that improves the generated Cypher query until it passes a validation check. The validation works by running EXPLAIN <query> on the generated query, and the query is accepted if it does not raise an error. It also ensures that the answer is not a hard-coded string, e.g., RETURN “Cannot answer”. The self-refinement loop is run a maximum of 3 times, after which it rejects the query.

3.3 Task 2: Caching and Performance

LRU Cache Design. We implement caching at two stages of the pipeline using the cachetools.LRUCache data structure with a maximum capacity of 128 entries each. The first cache stores schema pruning results, avoiding repeated LLM calls when the same question is asked against the same graph schema. The second cache stores generated Cypher queries, eliminating redundant query generation for previously seen question-schema combinations.

Cache keys are computed as a composite of question and schema hashes. Specifically, we compute the MD5 hash of the question

text and the MD5 hash of the serialized schema, then concatenate the first 8 hexadecimal characters of each to form a key in the format `<q_hash>_<s_hash>`. This design ensures that identical questions produce cache hits only when the underlying schema is also unchanged, which is important for correctness when the graph structure evolves. The caches are scoped to each GraphRAG instance rather than shared globally, which prevents warm-cache effects from contaminating benchmark comparisons across different enhancement configurations.

Benchmarking Infrastructure. [TODO: Describe the benchmarking setup, how timing data is collected, and how results are aggregated.]

Memory Measurement. [TODO: Describe memory measurement methodology if applicable.]

4 Performance Evaluation

4.1 Task 1: Text2Cypher Accuracy and Latency

The performance of the system is evaluated using a benchmarking suite that runs a set of prompts on the system with 8 different enhancement configurations. These prompts cover different kinds of queries, including entity lookup, multi-hop reasoning, boolean constraints, and schema disambiguation. The following prompts were used for the benchmarking suite:

- Who won the Nobel Prize in Physics in 2020?
- Which scholars won prizes in Physics and were affiliated with University of Cambridge?
- How many Nobel Prizes have been awarded in Chemistry?
- List all Nobel Prize winners from Sweden.
- Find all scholars whose name contains 'Einstein'.
- Who won the Nobel Prize in Physics in 2020?
- List Physics Nobel Prize winners located in Paris.
- Which laureates winning literature prizes are affiliated with Stanford but not Cambridge?
- Find Nobel laureates who are affiliated with both MIT and Harvard.
- List Nobel laureates born in Sweden and located in Paris.

Table 1 presents the ablation study showing accuracy-latency trade-offs. The Few-shot + Post-processing setting provided the best overall balance, achieving 100% success and syntax validity, 90% semantic correctness, and the lowest average latency (1.43 s). The All refinements configuration performed similarly but at a slightly slower average latency (1.61 s), which suggests that the additional refinement steps add overhead without improving the quality of the query's semantics.

The baseline configuration was clearly the worst, as it achieved only 80% across all correctness metrics and was more than three times slower than the best performing configurations. The self refinement only configuration was also inefficient, since while it maintained 100% success and validity with 90% semantic correctness, its average latency rose to 11.32 seconds. This is probably because of it running the expensive query generation multiple times.

All configurations except the baseline were able to produce a semantically correct query for 9 out of the 10 prompts. The only query that proved difficult was "List Nobel laureates born in Sweden and

located in Paris." This is because the generated queries either connect Paris to the birth city or use a union instead of an intersection, and they don't include the expected edge `is_located_in`.

4.2 Task 2: Performance and Efficiency

Table 2 shows the pipeline stage latency breakdown. It was generated using a separate script that aggregates the data from the benchmarking suite. From the table, we can see that there is a significant difference between cold runs and cache hits. Schema pruning, which is normally the slowest step, drops from about 1.45 seconds to essentially zero when cached. Query generation shows a similar pattern, falling from 121.7 ms to 0.1 ms. Response generation also becomes much faster, though not fully eliminated, and query execution sees only a small improvement. Together, these changes reduce end-to-end latency from 1.77 seconds to just 6.5 ms on cache hits.

In cold runs, schema pruning is the clear bottleneck, as it is responsible for more than 80% of the total latency. Response generation and query synthesis are next, while database execution remains negligible. However, on cache hits, the effect of pruning and query generation on the latency pretty much disappear, and the remaining time comes mostly from lightweight response formatting and a few milliseconds of query execution.

5 Discussion

- Key findings from ablation study
- Optimal configuration recommendation
- Challenges encountered and solutions
- Lessons learned

6 Conclusion

- Summary of Task 1 and Task 2 achievements
- Key contributions
- Practical insights

References

- [1] Daniel Granström, Joonatan Korpela, and Mikael Siidorow. 2025. Enhancing LLM Inference with GraphRAG. <https://github.com/dcgran/CS-E4780-project>. Accessed: 2025-11-24.
- [2] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A. Rossi, Subhabrata Mukherjee, Xianfeng Tang, Qi He, Zhigang Hua, Bo Long, Tong Zhao, Neil Shah, Amin Javari, Yinglong Xia, and Jiliang Tang. 2025. Retrieval-Augmented Generation with Graphs (GraphRAG). arXiv:2501.00309 [cs.IR] <https://arxiv.org/abs/2501.00309>

Table 1: Ablation Study - Accuracy vs Latency Trade-offs

Configuration	Syntax Correct (%)	Semantic Correct (%)	Avg Latency (ms)	Avg Iterations
Baseline (static few-shot)	80	80	4728	-
Only Few-shot selection	100	90	6494	-
Only Self-refinement	100	90	11316	1.0
Only Post-processing	90	90	1600	-
Few-shot + Post-processing	100	90	1430	-
Few-shot + Refinement	100	90	2043	0.9
Refinement + Post-processing	100	90	2189	1.0
All enhancements	100	90	1610	0.9

Table 2: Pipeline Stage Latency Breakdown With All Refinements (ms)

Stage	Without Cache	With Cache Hit
Schema Pruning	1447,3	0,0
Text2Cypher	121,7	0,1
Query Execution	4,4	2,6
Response Generation	199,0	3,8
Total	1772,4	6,5